

LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA

(JOBSHEET 7)



Disusun Oleh:

Nama : Masando Fami Ramadhan
NIM : 254107060011
Kelas : 1B
No. Absen : 14

PROGRAM STUDI SISTEM INFORMASI BISNIS
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2026

Daftar Isi

1. Tujuan	3
2. Praktikum	3
2.1. Percobaan 1: Sequential Search	3
2.1.1. Langkah-langkah	4
2.1.2. Pertanyaan	8
2.2. Percobaan 1: Sequential Search	9
2.2.1. Langkah-langkah	9
2.2.2. Pertanyaan	12

1. Tujuan

- Menjelaskan mengenai algoritma Searching.
- Membuat dan mendeklarasikan struktur algoritma Searching.
- Menerapkan dan mengimplementasikan algoritma Searching.

2. Praktikum

2.1. Percobaan 1: Sequential Search

Perhatikan diagram class Mahasiswa di bawah ini. Diagram class ini yang selanjutnya akan dibuat sebagai acuan dalam membuat kode program class Mahasiswa.

Mahasiswa
nim: String nama: String kelas: String ipk: double
Mahasiswa() Mahasiswa(nm: String, name: String, kls: String, ip: double) tampilInformasi(): void

Berdasarkan class diagram di atas, akan dibuat class Mahasiswa yang berfungsi untuk membuat objek mahasiswa yang akan dimasukkan ke dalam sebuah array. Terdapat sebuah konstruktor berparameter dan juga fungsi `tampilInformasi()` untuk menampilkan semua atribut yang ada.

MahasiswaBerprestasi
listMhs: Mahasiswa[5] idx: int
tambah(mhs: Mahasiswa): void tampil(): void sequentialSearch(double cari): int tampilPosisi(double x, int pos): void tampilDataSearch(double x, int pos) : void

Selanjutnya class diagram di atas merupakan representasi dari sebuah class yang berfungsi untuk melakukan operasi-operasi dari objek array Mahasiswa, misalkan untuk menambahkan objek mahasiswa, menampilkan semua data mahasiswa, untuk melakukan pencarian berdasarkan IPK menggunakan algoritma Sequential Search, menampilkan posisi dari data yang dicari, serta menampilkan data mahasiswa yang dicari.

2.1.1. Langkah-langkah

1. Tambahkan fungsi `sequentialSearching`, `tampilPosisi`, dan `tampilDataSearch` di file `MahasiswaBerprestasi{NoAbsen}.java` :

```
int sequentialSearching(double cari) {
    int posisi = -1;
    for (int j = 0; j < listMhs.length; j++) {
        if (listMhs[j].ipk == cari) {
            posisi = j;
            break;
        }
    }
    return posisi;
}

void tampilPosisi(double x, int pos) {
    if (pos != -1) {
        System.out.println("data mahasiswa dengan IPK : " + x + " ditemukan pada indeks " + pos);
    } else {
        System.out.println("data " + x + " tidak ditemukan");
    }
}

void tampilDataSearch(double x, int pos) {
    if (pos != -1) {
        System.out.println("nim\t : " + listMhs[pos].nim);
        System.out.println("nama\t : " + listMhs[pos].nama);
        System.out.println("kelas\t : " + listMhs[pos].kelas);
        System.out.println("ipk\t : " + x);
    } else {
        System.out.println("Data mahasiswa dengan IPK " + x + " tidak ditemukan");
    }
}
```

2. Modifikasi file `MahasiswaDemo{NoAbsen}.java` seperti idxmin:

```
import java.util.Scanner;
```

```

public class MahasiswaDemo14 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String nim, nama, kelas;
        double ipk;

        MahasiswaBerprestasi14 list = new MahasiswaBerprestasi14();

        System.out.println("SISTEM MANAJEMEN DATA MAHASISWA BERPRESTASI\n");
        for (int i = 0; i < list.listMhs.length; i++) {
            System.out.println("Masukkan data mahasiswa ke-"+(i+1));

            System.out.print("NIM : ");
            nim = sc.nextLine();

            System.out.print("Nama : ");
            nama = sc.nextLine();

            System.out.print("Kelas : ");
            kelas = sc.nextLine();

            System.out.print("IPK : ");
            ipk = sc.nextDouble();
            sc.nextLine();

            list.tambah(new Mahasiswa14(nim, nama, kelas, ipk));

            System.out.println("-----");
        }

        System.out.println("Data mahasiswa sebelum sorting: ");
        list.tampil();

        System.out.println("Pencarian data");
        System.out.println("-----");
        System.out.println("masukkan ipk mahasiswa yang dicari: ");
        System.out.print("IPK: ");
        double cari = sc.nextDouble();

        System.out.println("menggunakan sequential searching");
        double posisi = list.sequentialSearching(cari);
        int pss = (int) posisi;
        list.tampilPosisi(cari, pss);
        list.tampilDataSearch(cari, pss);

        sc.close();
    }
}

```

3. *Compile* dan *run* program tersebut.

```
Data mahasiswa sebelum sorting:
Nama: Rosie Wells
NIM: 2540000000001
Kelas: SIB 1A
IPK: 3.35
-----
Nama: Isabelle Aguilar
NIM: 2540000000002
Kelas: TI 1F
IPK: 3.74
-----
Nama: Julian Brewer
NIM: 2540000000003
Kelas: TI 2B
IPK: 3.33
-----
Nama: Joseph Aguilar
NIM: 2540000000004
Kelas: SIB 1B
IPK: 3.82
-----
Nama: Isabella Bryant
NIM: 2540000000005
Kelas: SIB 1D
IPK: 3.49
-----
Pencarian data
-----
masukkan ipk mahasiswa yang dicari:
IPK: menggunakan sequential searching
data mahasiswa dengan IPK : 3.49 ditemukan pada indeks 4
nim      : 2540000000005
nama     : Isabella Bryant
kelas    : SIB 1D
ipk      : 3.49
```


2.1.2. Pertanyaan

1. Jelaskan perbedaan method `tampilDataSearch` dan `tampilPosisi` pada class `MahasiswaBerprestasi`!

- `tampilPosisi` berfungsi untuk menampilkan **pesan posisi index** sesuai dengan data parameter yang dimasukkan. Sedangkan `tampilDataSearch` berfungsi untuk menampilkan detail data berdasarkan parameter yang dimasukkan. Jika pada parameter pos isinya **-1**, maka kedua fungsi yang menampilkan pesan data tidak ditemukan.

2. Jelaskan fungsi `break` pada kode program di bawah ini!

```
if (listMhs[j].ipk == cari) {  
    posisi = j;  
    break;  
}
```

- Break berfungsi untuk menghentikan loop apabila data telah ditemukan.
3. Apa fungsi variabel pos atau indeks hasil pencarian dalam program sequential search?
- Variabel tersebut berfungsi untuk menyimpan **data indeks** dari hasil *sequential search*. Jika tidak ditemukan, maka nilainya akan diset menjadi **-1**.
4. Jika terdapat lebih dari satu data dengan nilai yang sama, hasil pencarian sequential search yang dibuat di atas akan menampilkan data ke berapa? Jelaskan!
- Algoritma *sequential search* akan mencari **kemunculan pertama** dari suatu data apabila data tersebut terdapat lebih 1. Hal ini disebabkan karena *sequential search* melakukan pencarian secara berurutan (dari indeks 0 sampai index ke n)
5. Berkaitan dengan pertanyaan nomor 2 di atas, apa yang terjadi jika perintah break dihapus dari kode di atas?
- Walaupun data sudah ditemukan, fungsi tersebut akan tetap melakukan pencarian sampai elemen terakhir. Sehingga apabila data yang dicari adalah data duplikat, data indeks yang dikembalikan akan menunjuk ke **kemunculan terakhir** data tersebut.

2.2. Percobaan 1: Sequential Search

2.2.1. Langkah-langkah

1. Tambahkan fungsi `findBinarySearch`, pada file `MahasiswaBerprestasi{NoAbsen}.java`:

```
int findBinarySearch(double cari, int left, int right) {
    int mid;
    if (right >= left) {
        mid = (left + right) / 2;
        if (cari == listMhs[mid].ipk) {
            return (mid);
        } else if (listMhs[mid].ipk > cari) {
            return findBinarySearch(cari, left, mid - 1);
        } else {
            return findBinarySearch(cari, mid + 1, right);
        }
    }
    return -1;
}
```

2. Tambahkan baris berikut di file `MahasiswaDemo{NoAbsen}.java` seperti idxmin:

```
System.out.println("-----");
System.out.println("Pencarian data");
System.out.println("-----");
System.out.println("masukkan ipk mahasiswa yang dicari: ");
System.out.print("IPK: ");
cari = sc.nextDouble();

System.out.println("menggunakan binary search");
double posisi2 = list.findBinarySearch(cari, 0, list.listMhs.length-1);
int pss2 = (int) posisi2;
list.tampilPosisi(cari, pss2);
list.tampilDataSearch(cari, pss2);
```

3. *Compile* dan *run* program tersebut.

```
Kelas: TI 1A
IPK: 3.74
-----
Nama: Julian Brewer
NIM: 2540000000003
Kelas: TI 2B
IPK: 3.33
-----
Nama: Joseph Aguilar
NIM: 2540000000004
Kelas: SIB 1B
IPK: 3.82
-----
Nama: Isabella Bryant
NIM: 2540000000005
Kelas: SIB 1D
IPK: 3.49
-----
Pencarian data
-----
masukkan ipk mahasiswa yang dicari:
IPK: menggunakan sequential searching
data mahasiswa dengan IPK : 3.35 ditemukan pada
indeks 0
nim      : 2540000000001
nama     : Rosie Wells
kelas    : SIB 1A
ipk      : 3.35
-----
Pencarian data
-----
masukkan ipk mahasiswa yang dicari:
IPK: menggunakan binary search
data 3.35 tidak ditemukan
Data mahasiswa dengan IPK 3.35 tidak ditemukan
```


2.2.2. Pertanyaan

1. Tunjukkan pada kode program yang mana proses *divide* dijalankan!
 - Proses **divide** terjadi pada baris `mid = (left + right) / 2;`, di mana rentang pencarian dibagi menjadi dua bagian melalui titik tengah.
2. Tunjukkan pada kode program yang mana proses *conquer* dijalankan!
 - Proses **conquer** terjadi saat pemanggilan rekursif `findBinarySearch(cari, left, mid - 1)` atau `findBinarySearch(cari, mid + 1, right)`, di mana program fokus mencari pada salah satu sub-bagian yang relevan.
3. Apa fungsi `left`, `right`, dan `mid`?
 - `left`: Indeks awal rentang pencarian.
 - `right`: Indeks akhir rentang pencarian.
 - `mid`: Indeks titik tengah yang digunakan sebagai pembanding dengan data yang dicari.
4. Jika data IPK yang dimasukkan tidakurut. Apakah program masih dapat berjalan? Mengapa demikian?
 - Program tetap berjalan, tetapi hasilnya tidak akurat (salah). Hal ini dikarenakan algoritma **binary search** hanya dapat bekerja dengan syarat data **telah terurut**.
5. Jika IPK yang dimasukkan dari IPK terbesar ke terkecil (misal: 3.8, 3.7, 3.5, 3.4, 3.2) dan elemen yang dicari adalah 3.2. Bagaimana hasil dari binary search? Apakah sesuai? Jika tidak sesuai maka ubahlah kode program binary search agar hasilnya sesuai
 - Hasilnya tidak sesuai karena logika default pada fungsi `findBinarySearch` adalah untuk data yang telah terurut secara *ascending*. Oleh karena itu, kita perlu memanggil fungsi `selectionSort` sebelum fungsi `findBinarySearch` dipanggil:

```
System.out.println("menggunakan binary search");
list.selectionSort(); // Sort terlebih dahulu
double posisi2 = list.findBinarySearch(cari, 0, list.listMhs.length-1);
int pss2 = (int) posisi2;
list.tampilPosisi(cari, pss2);
list.tampilDataSearch(cari, pss2);
```

6. Jelaskan bagaimana binary search menentukan bahwa data yang dicari tidak ditemukan di dalam array.
- Data dianggap tidak ditemukan jika kondisi `right >= left` tidak lagi terpenuhi (indeks `left` melewati indeks `right`), sehingga fungsi keluar dari blok `if` dan mengembalikan nilai `-1`.
7. Modifikasi program di atas yang mana jumlah mahasiswa yang diinputkan sesuai dengan masukan dari keyboard.
- **MahasiswaDemo14.java**

```
...
System.out.println("SISTEM MANAJEMEN DATA MAHASISWA BERPRESTASI\n");

// Input jml sebelum input
System.out.print("Jumlah mahasiswa: ");
int jml = sc.nextInt();
sc.nextLine();

MahasiswaBerprestasi14 list = new MahasiswaBerprestasi14(jml);
...
```

MahasiswaBerprestasi14.java

```
...
// Tambah constructor default tanpa parameter & dengan parameter
MahasiswaBerprestasi14() {
    this.listMhs = new Mahasiswa14[5];
}

MahasiswaBerprestasi14(int len) {
    this.listMhs = new Mahasiswa14[len];
}
...
```

Link Praktikum: https://github.com/fami0110/Praktikum_ASU